Asteroids: Data and Architecture Design

Ahmet Sevinç: 6962084
Bram den otter: 7153651
6 November, 2026

1. Introduction and Rules

This document describes the implementation of the Asteroids game. Asteroids is a 2D game where the player controls a spaceship that must destroy asteroids and possible hostile enemies on the screen to increase their score. Furthermore, the player must also avoid being hit themselves to obtain a high score.

2. Design

2.1 Data Structures

2.1.1 The Game State

The full state of the game is represented using a GameState data type:

```
data GameState = GameState { player :: Player,
                             asteroids :: [Asteroid],
                             enemiesExplo :: [Animation],
                             bullets :: [Bullet],
                             keysPressed :: KeysPressed,
                             elapsedTime :: ElapsedTime,
                             score :: Score,
                             highScore :: HighScore,
                             gameState :: State,
                             }
```

2.1.2 The Player and Playing Field

The movement, player, enemies, bullets and the rest will be presented by an enumeration type:

Data Player = Player Picture Point Speed Direction Lives
Data Variant = Passive | Chase
Data Bullet = Bullet Picture Point Direction Speed
Data Asteroid = Asteroid Picture Point Direction Speed Variant
Data Animation = Animation (Float, Float) Float Point [Bool]
Data State = Start | Pause | Gameover

The lives, scores and speed will be int, direction a float:
Type Speed = Int
Type Direction = Float
Type Score = Int
Type Lives = Int

2.1.3 Player Movement

The player can only move forward at a fixed speed, but the direction changes using the left and right arrow keys. This is done by a function:
movePlayer :: Player -> Float -> Player

The movements will be with “w, a, d” or “Up, Left, Right” arrows.

2.2 Enemies

To make the game fair to play, whenever an asteroid spawns in, it should spawn a certain distance away from the player, so that the player doesn’t just lose by an unfortunate spawn. Or another approach that could be considered, is the asteroids spawning just outside the game window borders. This’ll incentivize the player to mind their positioning, which adds to the strategy.
Certain asteroids will appear after a while, which will chase the player’s location instead of having a randomized direction.
A simple animation will play if the asteroid gets destroyed

2.3 Interface

The player will interact with the game with their keyboard and a separate game window.

2.4 Implementation of the Minimum Requirements

Player: The player controls their spaceship

Enemies: The program controls the asteroids. As explained in 2.2, there will be a normal enemy unit and a smart enemy unit.

Randomness: The enemy appears in random positions when they spawn.

Pause: The game can be paused by pressing the ‘p’ key.

Interaction with the file system: Highscores will be saved in a text file. The highscores will also be shown at the end of a game.